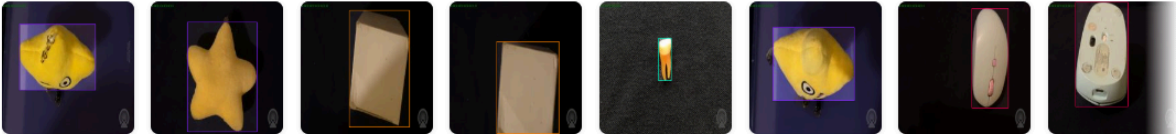


Imágenes tomadas de la práctica anterior y etiquetadas en roboflow

Uses Roboflow Credits [View usage ↗](#)

How to Upload Custom Weights

252 Total Images [View All Images →](#)



Dataset Split

TRAIN SET
177 Images
78%

VALID SET
50 Images
20%

TEST SET
25 Images
10%

Preprocessing

Auto-Orient: Applied
Resize: Stretch to 512x512

Increases

No augmentations were applied

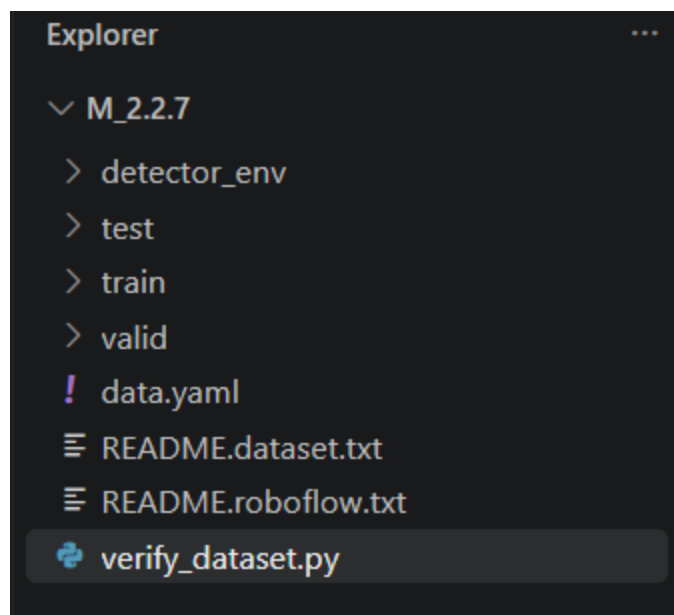
Imagen 1. Etiquetado en Rovoflow

Antes de comenzar el entrenamiento se ejecutó el archivo `verify_dataset.py`. El programa permitió comprobar que las imágenes del conjunto de entrenamiento tuvieran su archivo de etiquetas correspondiente.

```
(detector_env) PS C:\Users\maria\OneDrive\Documentos\M_2.2.7> & c:\Users\maria\OneDrive\Documentos\M_2.2.7\detector_env\Scripts\python.exe c:/Users/maria/OneDrive/Documentos/M_2.2.7/verify_dataset.py
Images without a label file: 0
(detector_env) PS C:\Users\maria\OneDrive\Documentos\M_2.2.7> |
```

Imagen 2. Images without a label file: 0

La verificación fue importante porque permitió comprobar que el conjunto de datos estuviera correctamente preparado antes de iniciar el entrenamiento.



Para el entrenamiento se utilizó el modelo YOLO11n, utilizando el archivo data.yaml con las cinco clases. El entrenamiento fue configurado inicialmente para 50 épocas. Durante el primer intento, el entrenamiento llegó aproximadamente a la época 40 y se detuvo debido a un problema de memoria de la GPU. Para solucionarlo se redujo el tamaño del lote (batch) a 4 y se continuó el entrenamiento desde el archivo last.pt.

Esto permitió completar correctamente las 50 épocas.

```
Remapped 1/5 cls head rows from pretrained weights by class name
Transferred 451/499 items from pretrained weights
Freezing layer 'model.23.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: downloading yolo26n.pt for AMP checks (one-time, not used for training)...
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0...: 100% 5.3MB 2.4MB/s 2.2s
AMP: checks passed
WARNING train: Slow image access detected (ping: 0.20.0 ms, read: 30.17.4 MB/s, size: 18.1 KB). Use local storage
instead of remote/mounted storage for better performance. See https://docs.ultralytics.com/guides/model-training-tips
train: Scanning C:\Users\maria\OneDrive\Documents\M_2.2.7\train\labels.: 100% 177/177 1.2Kit/s 0.1s
train: New cache created: C:\Users\maria\OneDrive\Documents\M_2.2.7\train\labels.cache
WARNING val: Slow image access detected (ping: 0.30.2 ms, read: 21.411.2 MB/s, size: 20.4 KB). Use local storage
instead of remote/mounted storage for better performance. See https://docs.ultralytics.com/guides/model-training-tips
val: Scanning C:\Users\maria\OneDrive\Documents\M_2.2.7\valid\labels.....: 100% 50/50 1.1Kit/s 0.0s
val: New cache created: C:\Users\maria\OneDrive\Documents\M_2.2.7\valid\labels.cache
optimizer: 'optimizer-auto' found, ignoring 'lr0=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr0' and 'momentum' automatically...
optimizer: AdamW(lr=0.001111, momentum=0.9) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
Plotting labels to C:\Users\maria\OneDrive\Documents\M_2.2.7\runs\detect\train\labels.jpg...
Using 177 train, 50 val images for fraction=1.0 at imgsz=640
Using 8 dataloader workers
Logging results to C:\Users\maria\OneDrive\Documents\M_2.2.7\runs\detect\train
Starting training for 50 epochs...
```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/50	2.3G	0.8924	3.901	1.393	3	100% 12/12 1.9s/it 23.4s
	Class	Images	Instances	Box(P	R	mAP.: 100% 2/2 1.1s/it 2.3s
	all	50	51	0.00286	0.625	0.218 0.167
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/50	2.89G	0.803	3.469	1.252	4	100% 12/12 1.8it/s 6.8s
	Class	Images	Instances	Box(P	R	mAP.: 100% 2/2 5.1it/s 0.4s
	all	50	51	0.0121	0.8	0.393 0.284
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/50	2.3G	0.8924	3.901	1.393	3	...: 100% 12/12 1.9s/it 23.4s
	Class	Images	Instances	Box(P	R	mAP...: 100% 2/2 1.1s/it 2.3s
	all	50	51	0.00286	0.625	0.218 0.167
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/50	2.85G	0.803	3.469	1.252	4	...: 100% 12/12 1.8it/s 6.8s
	Class	Images	Instances	Box(P	R	mAP...: 100% 2/2 5.1it/s 0.4s
	all	50	51	0.0121	0.8	0.393 0.284
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/50	2.85G	0.9818	3.074	1.337	3	...: 100% 12/12 1.8it/s 6.7s
	Class	Images	Instances	Box(P	R	mAP...: 100% 2/2 5.7it/s 0.3s
	all	50	51	0.00519	0.775	0.376 0.25
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
4/50	2.85G	1.001	2.7	1.373	4	...: 100% 12/12 1.8it/s 6.7s
	Class	Images	Instances	Box(P	R	mAP...: 100% 2/2 6.0it/s 0.3s
	all	50	51	0.00387	0.733	0.479 0.302
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
5/50	2.85G	0.9463	2.224	1.367	3	...: 100% 12/12 1.8it/s 6.7s
	Class	Images	Instances	Box(P	R	mAP...: 100% 2/2 4.6it/s 0.4s
	all	50	51	0.958	0.177	0.506 0.403
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
6/50	2.85G	0.9435	1.975	1.428	1	...: 100% 12/12 1.8it/s 6.8s
	Class	Images	Instances	Box(P	R	mAP...: 100% 2/2 4.6it/s 0.4s
	all	50	51	0.529	0.42	0.563 0.417
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
7/50	2.85G	0.8923	1.847	1.284	2	...: 100% 12/12 1.8it/s 6.8s
	Class	Images	Instances	Box(P	R	mAP...: 100% 2/2 6.2it/s 0.3s
	all	50	51	0.828	0.518	0.651 0.518
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size

Imagen 3 y 4. Primer entrenamiento

```

all      50      51      0.938      0.928      0.905      0.806

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
39/50   2.85G    0.6031   0.9419   1.096     1          ...: 100% 12/12 1.8it/s 6.7s
      Class   Images  Instances  Box(P     R          mAP...: 100% 2/2 5.6it/s 0.4s
      all    50      51      0.909     0.927     0.91      0.801

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
40/50   2.85G    0.7659   0.8391   1.255     1          ...: 100% 12/12 1.8it/s 6.7s
      Class   Images  Instances  Box(P     R          mAP...: 100% 2/2 6.5it/s 0.3s
      all    50      51      0.95     0.878     0.904     0.784

Closing dataloader mosaic

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
: 0% 0/12 0.8s
Traceback (most recent call last):
  File "c:\Users\maria\OneDrive\Documentos\M_2.2.7\train.py", line 8, in <module>
    results = model.train(
              ^^^^^^^^^^^
  File "c:\Users\maria\OneDrive\Documentos\M_2.2.7\detector_env\Lib\site-packages\ultralytics\engine\model.py", line 871, in train
    self.trainer.train()
  File "c:\Users\maria\OneDrive\Documentos\M_2.2.7\detector_env\Lib\site-packages\ultralytics\engine\trainer.py", line 246, in train
    self._do_train()
  File "c:\Users\maria\OneDrive\Documentos\M_2.2.7\detector_env\Lib\site-packages\ultralytics\engine\trainer.py", line 507, in _do_train
    loss, self.loss_items = self.model(batch)
                             ^^^^^^^^^^^^^^^
  File "c:\Users\maria\OneDrive\Documentos\M_2.2.7\detector_env\Lib\site-packages\torch\nn\modules\module.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "c:\Users\maria\OneDrive\Documentos\M_2.2.7\detector_env\Lib\site-packages\torch\nn\modules\module.py", line 1791, in _call_impl
    return forward_call(*args, **kwargs)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```

all      50      51      0.928      0.878      0.888      0.785

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
49/50   0.762G   0.4676   0.8839   1.058     1          ...: 100% 45/45 7.1it/s 6.3s
      Class  Images  Instances  Box(P    R    mAP...: 100% 7/7 20.6it/s 0.3s
      all    50      51      0.928    0.878  0.891    0.79

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
50/50   0.762G   0.4733   0.8636   1.059     1          ...: 100% 45/45 7.3it/s 6.2s
      Class  Images  Instances  Box(P    R    mAP...: 100% 7/7 21.1it/s 0.3s
      all    50      51      0.932    0.878  0.893    0.794

10 epochs completed in 0.028 hours.
Optimizer stripped from C:\Users\maria\OneDrive\Documentos\M_2.2.7\runs\detect\train\weights\last.pt, 5.5MB
Optimizer stripped from C:\Users\maria\OneDrive\Documentos\M_2.2.7\runs\detect\train\weights\best.pt, 5.5MB

Validating C:\Users\maria\OneDrive\Documentos\M_2.2.7\runs\detect\train\weights\best.pt...
Ultralytics 8.4.142 Python-3.11.9 torch-2.11.0+cu128 CUDA:0 (NVIDIA GeForce RTX 4050 Laptop GPU, 6140MiB)
YOLO11n summary (fused): 100 layers, 2,583,127 parameters, 0 gradients, 6.4 GFLOPs
      Class  Images  Instances  Box(P    R    mAP...: 100% 7/7 1.7it/s 4.1s
      all    50      51      0.926    0.929    0.936    0.82
      Caja    10      11      0.911    0.909    0.905    0.737
      diente    8       8      0.878     1     0.926    0.802
      estrella  10      10      0.898     1     0.995    0.952
      mouse    10      10       1     0.904    0.995    0.835
      squishy  12      12      0.945    0.833    0.86     0.772
Speed: 0.7ms preprocess, 70.7ms inference, 0.0ms loss, 3.6ms postprocess per image

```

Imagen 5 y 6. Segundo entrenamiento a partir de 41 epocas

Después de completar el entrenamiento se realizó la validación del modelo.

Métrica	Resultado
Precisión	92.6%
Recall	92.9%
mAP50	93.6%
mAP50-95	82.0%

Estos resultados muestran que el modelo logró aprender las cinco clases y obtener un buen desempeño durante la validación.

Clase	mAP50
Caja	90.5%
Diente	92.6%
Estrella	99.5%
Mouse	99.5%
Squishy	86.0%

Las clases Estrella y Mouse obtuvieron los mejores resultados, ambas con un mAP50 de 99.5%. La clase Squishy obtuvo el resultado más bajo, con 86.0%, aunque continuó mostrando un desempeño aceptable.

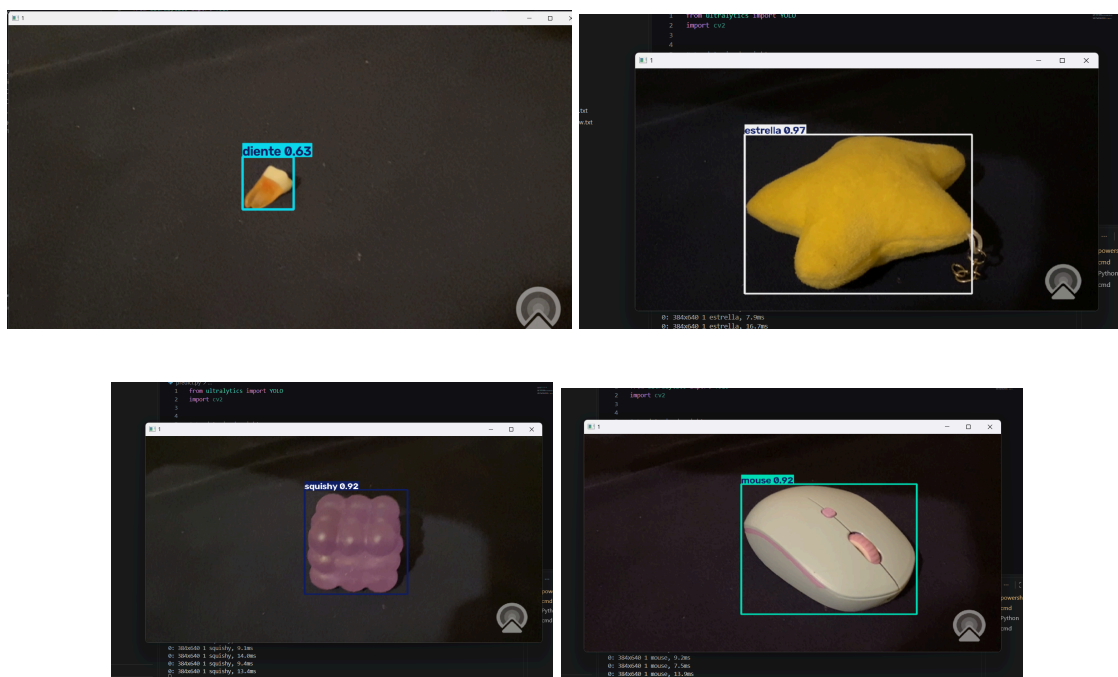
Después del entrenamiento se utilizó el archivo predict.py para realizar detecciones mediante la cámara web.

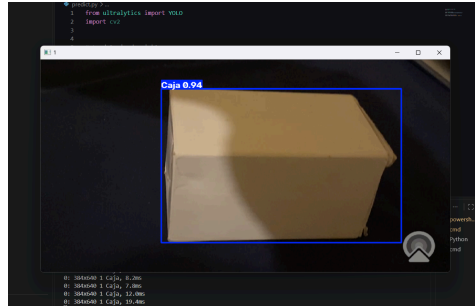
El modelo utilizó los pesos:

```
model = YOLO('runs/detect/train/weights/best.pt')
```

y posteriormente se inició la detección en tiempo real mediante:

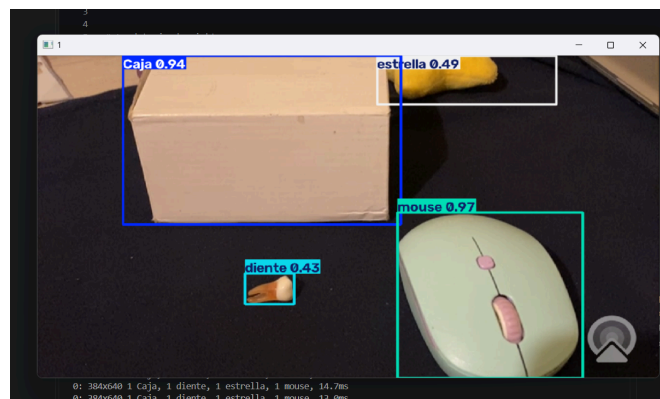
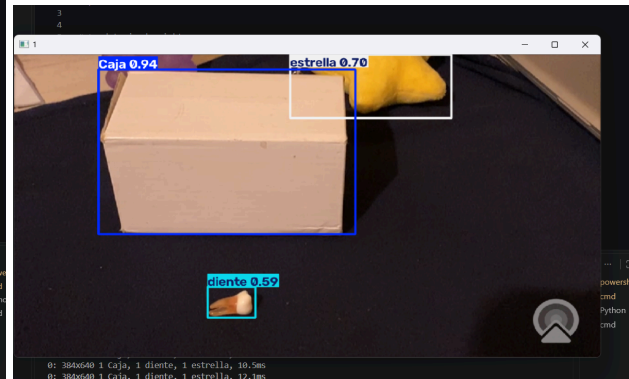
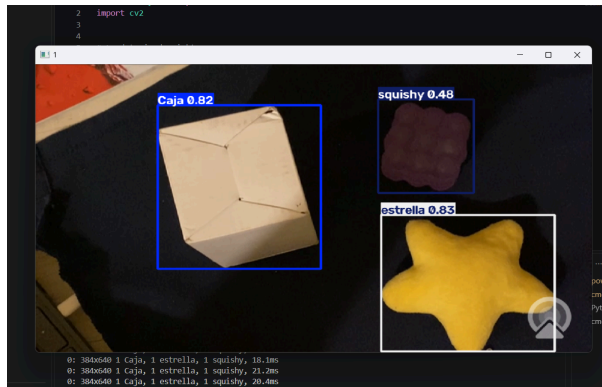
```
results = model.predict(source='0', show=True)
```





El modelo logró reconocer correctamente las cinco clases.

Además de realizar las pruebas individuales, se realizó una prueba colocando cuatro objetos diferentes frente a la cámara al mismo tiempo. El modelo logró identificar los objetos simultáneamente, mostrando que puede realizar varias detecciones dentro de una misma imagen.



Guided Experiments / Experimentos guiados

Keep the **Calibrated** preset (Image Size 640 px, Confidence Threshold 25%, Object Area 5000 px², Label Noise 10). Record the **Detection confidence readout** and the **System Accuracy metric readout**. — Why does a confident score clear a low threshold without hesitation?

Configuración:

- Image Size: 640 px
- Confidence Threshold: 25%
- Object Area: 5000 px²
- Label Noise: 10

Resultados:

- Detection confidence: 86.0%
- System Accuracy: 100.0%
- Frame verdict: DETECTED

R: El objeto se detecta correctamente porque la confianza obtenida fue de 86%, mientras que el umbral mínimo era de solamente 25%. Como la confianza supera ampliamente el umbral, el sistema acepta la detección.

Switch to **Low-res-miss: Small Input** (Image Size drops to 320 px). Record both readouts again. — Why does halving the input resolution shrink the apparent area enough for the model to lose a real object entirely?

Configuración:

- Image Size: 320 px
- Confidence Threshold: 25%
- Object Area: 5000 px²
- Label Noise: 10

Resultados:

- Frame verdict: MISSED
- System Accuracy: 0.0%

R: Al reducir la resolución de 640 a 320 píxeles, el objeto ocupa una menor área dentro de la imagen. Esto puede hacer que el objeto sea demasiado pequeño para que el modelo lo reconozca correctamente. Por eso, aunque el objeto está presente, el sistema puede no detectarlo.

Activate **Small-object: Distant Debris** (Object Area 1500 px²) while keeping the calibrated resolution. Record the **Frame verdict readout** and the **System Accuracy metric readout**. — Why is staying silent the correct behavior instead of raising a false detection?

Configuración:

- Image Size: 640 px
- Confidence Threshold: 25%
- Object Area: 1500 px²
- Label Noise: 10

Resultados:

- Frame verdict: SILENT
- System Accuracy: 100.0%

R: En este caso el objeto tiene un área de 1500 px², que está por debajo del límite de 2500 px² establecido por la simulación. Por lo tanto, no detectarlo es la decisión correcta. Permanecer en silencio evita que el sistema genere una detección falsa.

Apply **Noisy-strict: Depressed Confidence** (Label Noise 70, Confidence Threshold 85%). Compare the **Detection confidence readout** with experiment one. — How do mislabeled training boxes depress confidence, and why does demanding a stricter threshold compound the damage?

Configuración:

- Image Size: 640 px
- Confidence Threshold: 85%
- Object Area: 5000 px²
- Label Noise: 70

Resultados:

- Detection confidence: 26.0%
- Frame verdict: MISSED
- System Accuracy: 0.0%

R: El objeto debería ser detectado porque su área es de 5000 px², pero la confianza disminuyó hasta 26% debido al alto nivel de ruido en las etiquetas. Además, el umbral de confianza se estableció en 85%, por lo que el 26% no es suficiente para aceptar la detección. Como resultado, el sistema no detecta el objeto y se produce un falso negativo, por lo que la exactitud es de 0%.

Tabla de resultados

Experimento	Resolución	Umbral	Área	Ruido	Resultado
Calibrated	640 px	25%	5000 px ²	10	DETECTED
Low-res-miss	320 px	25%	5000 px ²	10	MISSED
Small-object	640 px	25%	1500 px ²	10	SILENT
Noisy-strict	640 px	85%	5000 px ²	70	SILENT

En esta práctica se logró extender el detector YOLO para trabajar con cinco clases: Caja, Diente, Estrella, Mouse y Squishy. Primero se preparó y verificó el conjunto de datos, obteniendo cero imágenes sin archivo de etiqueta. Después se realizó el entrenamiento del modelo YOLO11n durante 50 épocas y se obtuvo un mAP50 general de 93.6%.

Posteriormente, el modelo fue probado mediante la cámara web y logró reconocer correctamente las cinco clases. También se comprobó que podía detectar varios objetos al mismo tiempo. Los experimentos realizados en el simulador ayudaron a comprender que la resolución, el tamaño de los objetos, las etiquetas utilizadas durante el entrenamiento y el nivel de confianza pueden influir en las detecciones. En general, la práctica permitió comprobar de manera práctica cómo se prepara, entrena y utiliza un modelo YOLO para detectar diferentes objetos.